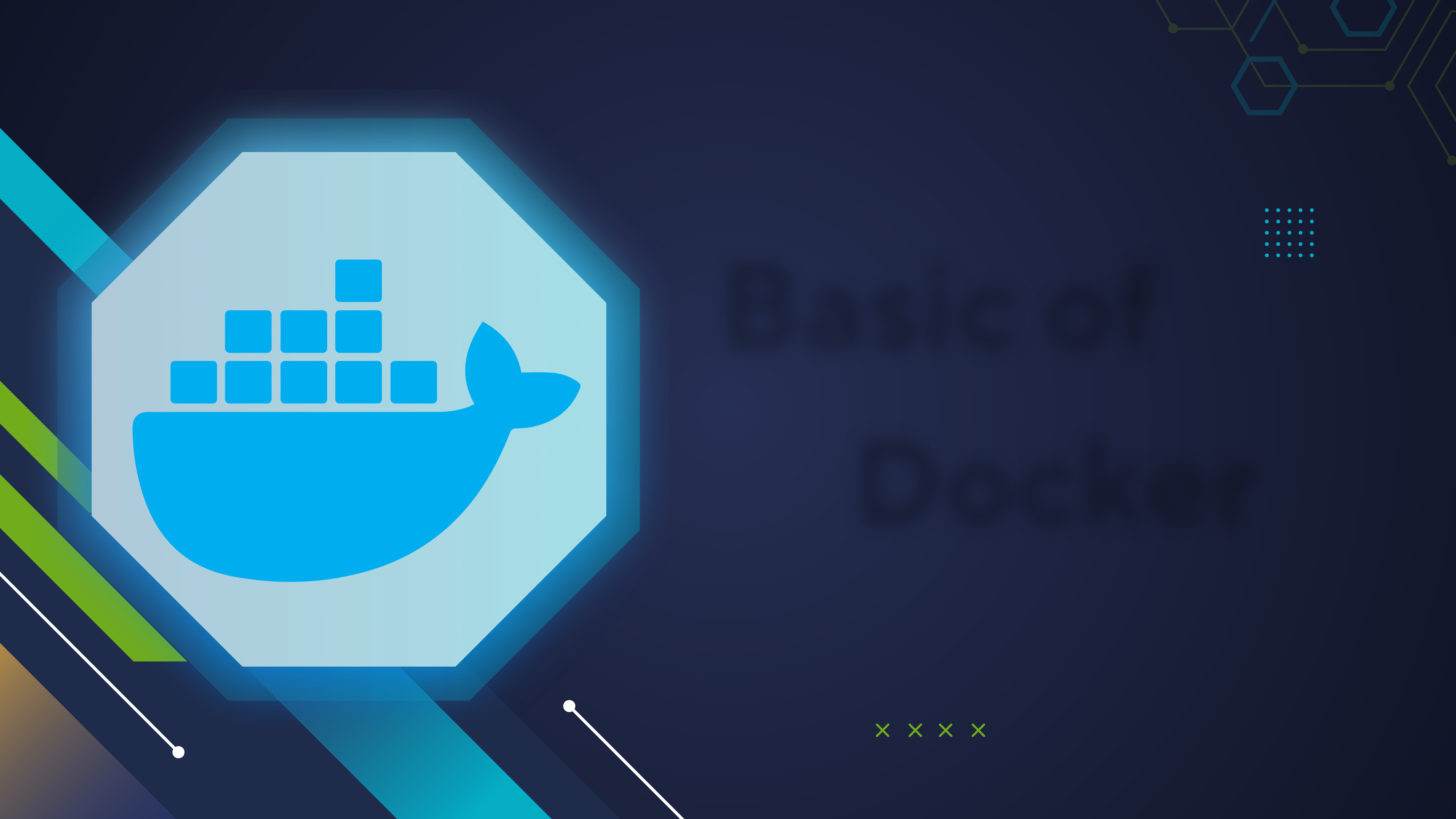


รายวิชา 06016418

SERVER-SIDE WEB DEVELOPMENT

การพัฒนาเว็บไซต์ฝั่งเซิร์ฟเวอร์





สุม

มัน Run ไ

แต่!!
บนค

× × × ×



"It works



- Environment Mismatch: ความแตกต่างระหว่างเครื่อง Dev และ Production
- Dependency Hell: ปัญหาไลบรารีคนละเวอร์ชัน หรือ OS Settings ที่ไม่ตรงกัน
- Manual Setup: การติดตั้งซอฟต์แวร์แบบเดิมใช้เวลานานและเสี่ยงต่อความผิดพลาด

x x x x



x x x x

ที่ทำให้ I

สา



Libraries

เวอร์ชันของ Node.js, Python หรือ Java ที่เครื่อง Dev อาจไม่เท่ากับบน Server



OS Conf

การตั้งค่า Environment Variables หรือ Permissions ของ Linux ที่ต่างกัน



Multi-Cloud

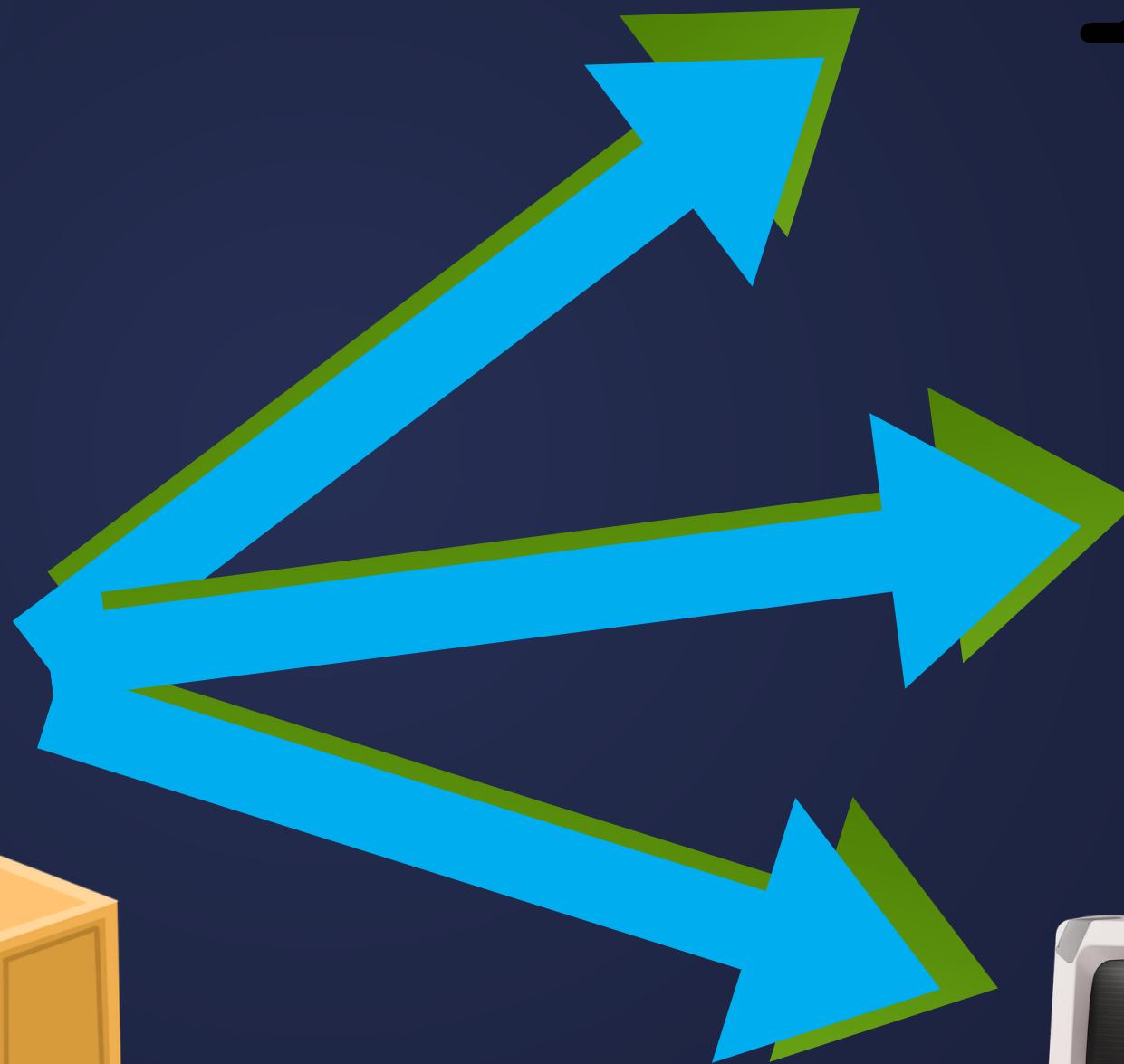
Port, Firewall หรือการเข้าถึงฐานข้อมูลที่เครื่องส่วนตัวทำได้ แต่ Server ติดขัด

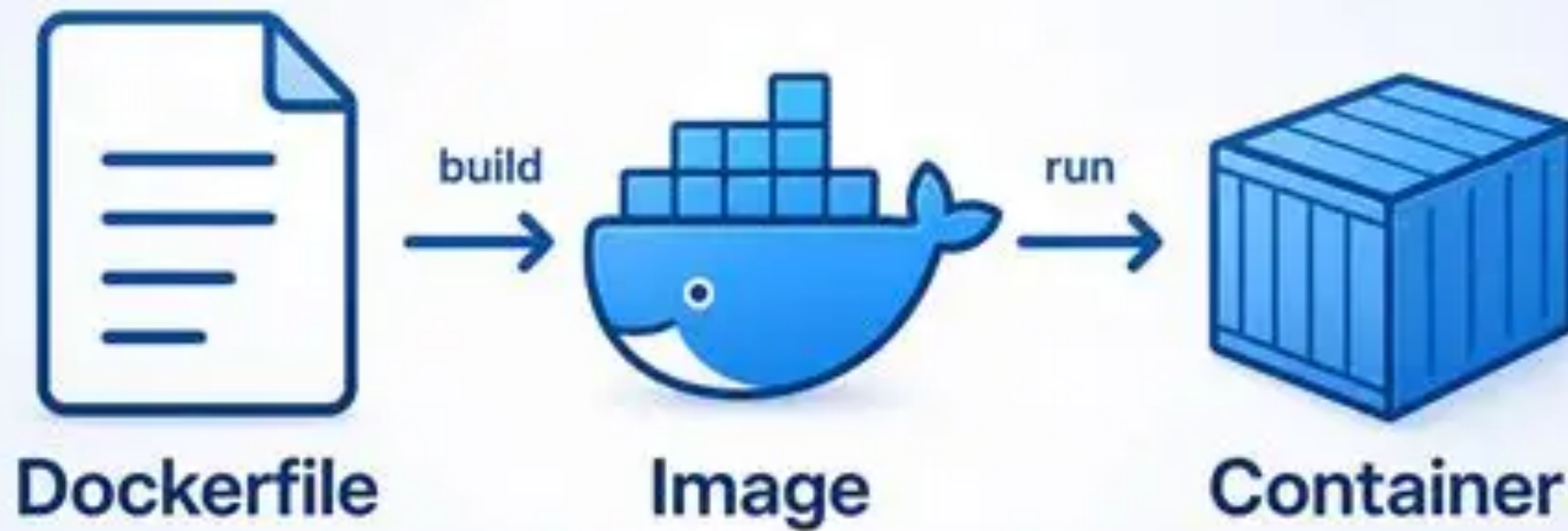


x x x x



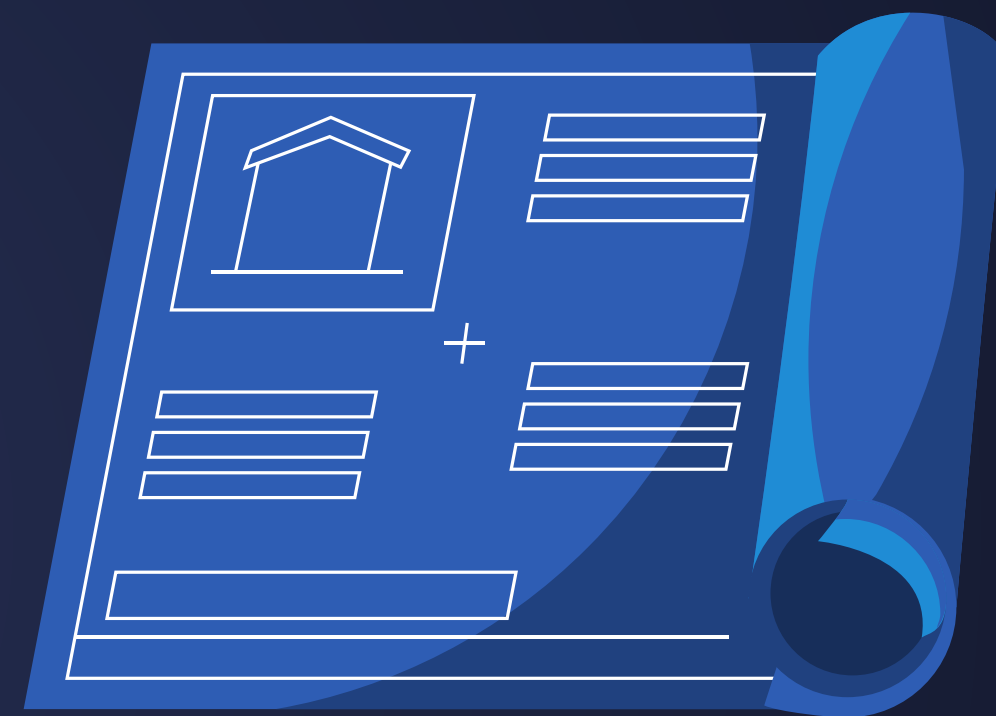
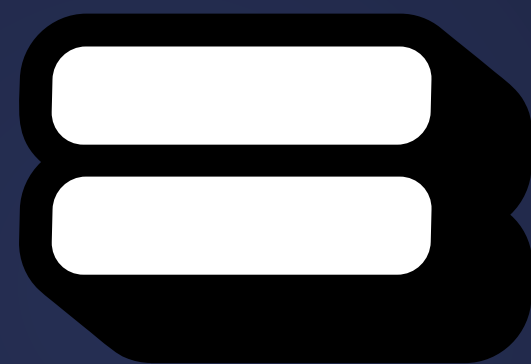
× × × ×





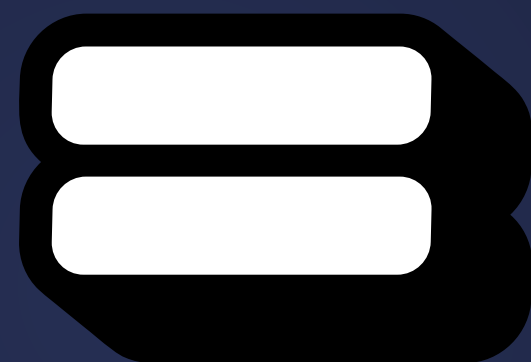
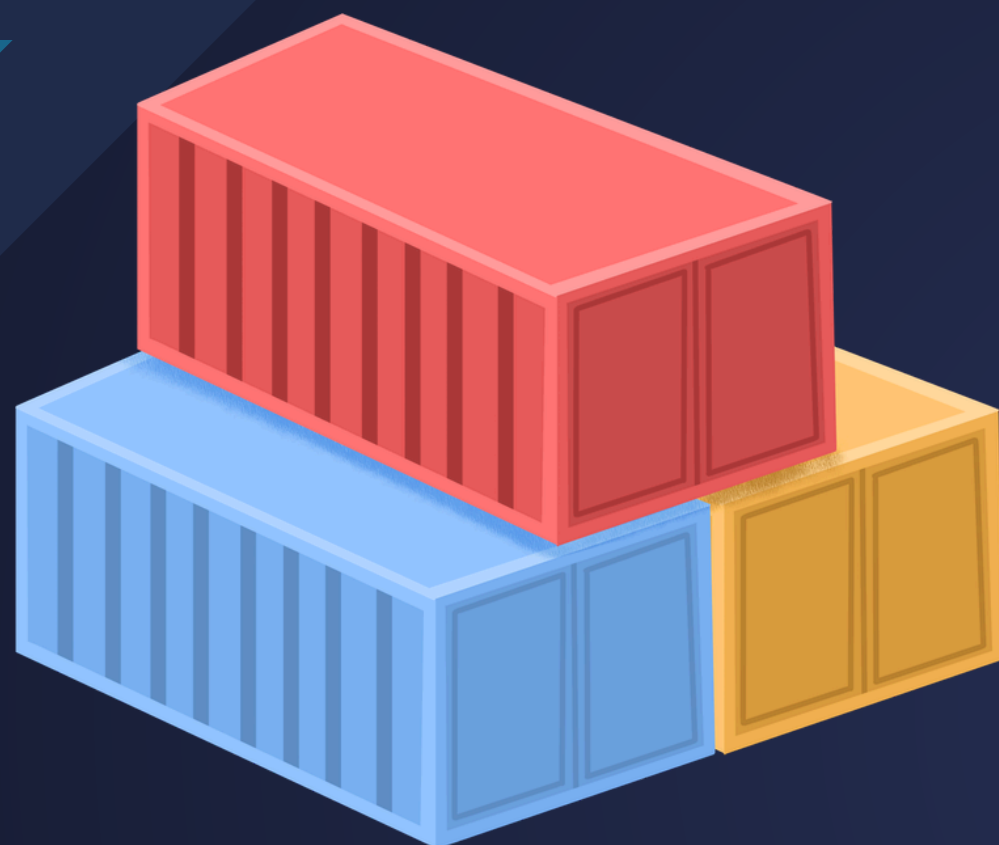
Docker เป็นเครื่องมือที่ช่วยให้นักพัฒนาสามารถ แยก และ บรรจุ Image แอปพลิเคชันที่พัฒนาขึ้นรวมถึงสภาพแวดล้อมต่าง ๆ ของการทำงานขึ้นมาได้ โดยเราจะเรียกว่า “คอนเทนเนอร์” (Containers)

x x x x



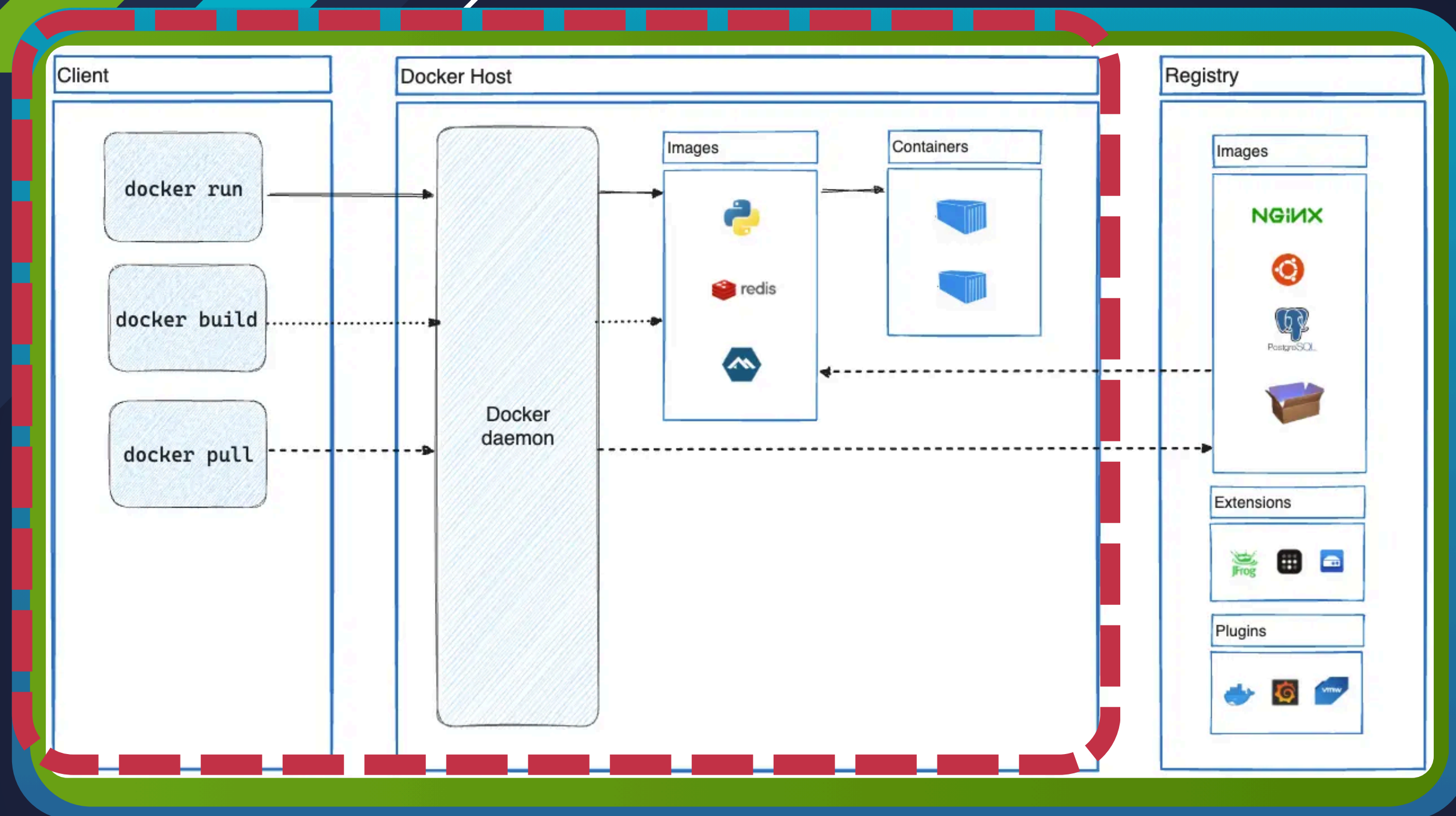
Docker Image: เปรียบเสมือน "พิมพ์เขียว" (Blueprint) หรือแผ่น CD ติดตั้ง
โปรแกรมที่เป็น Read-only

x x x x



Docker Container: เปรียบเสมือน "บ้านที่สร้างจากพิมพ์เขียว" คือตัวโปรแกรมที่กำลังทำงานจริง (Runtime instance)

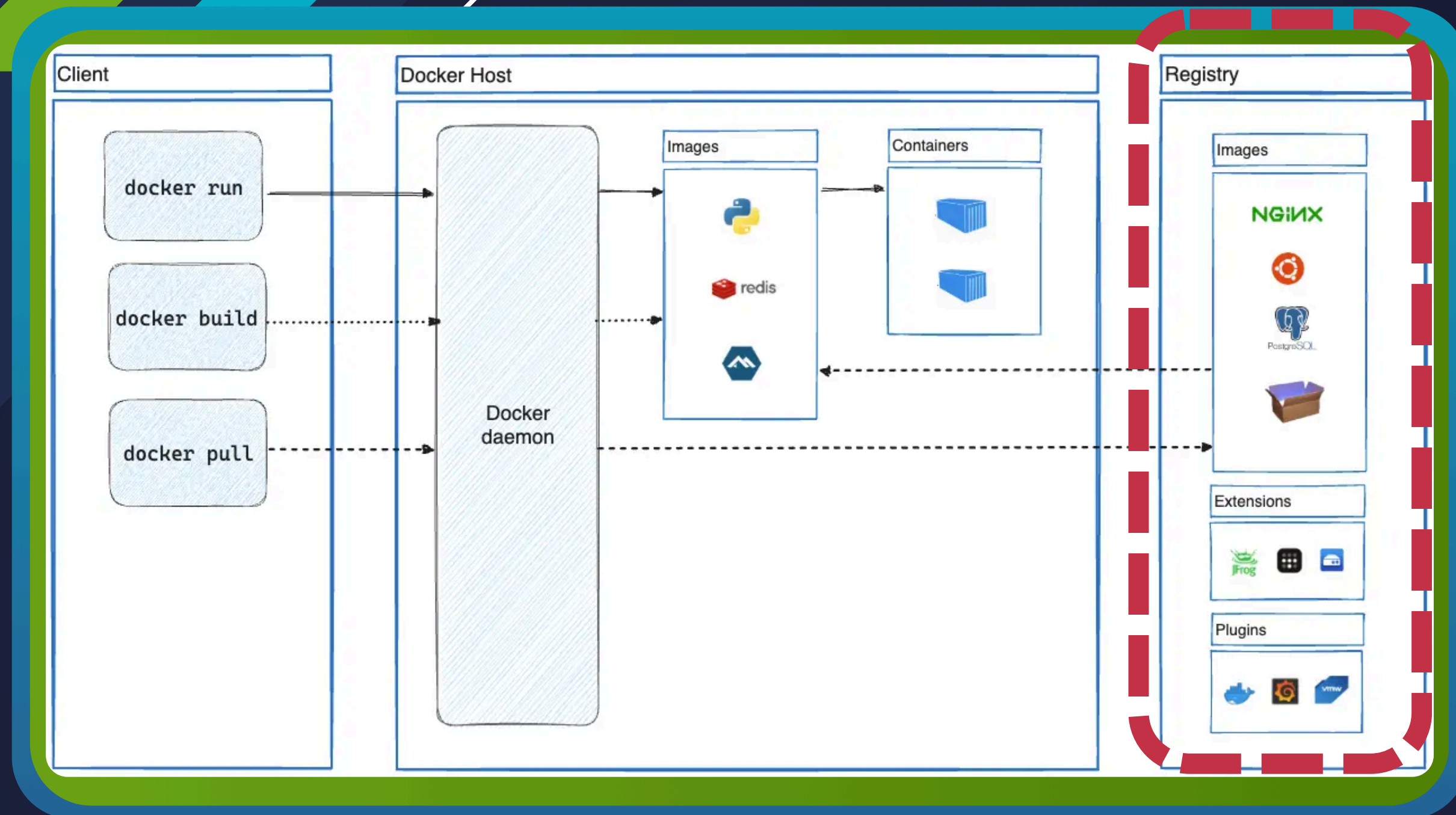
x x x x



โดยปกติแล้ว **Docker** ประกอบด้วย:

Docker Daemon: ทำหน้าที่จัดการสร้าง การทำงาน และการกระจายคอนเทนเนอร์

Docker Client: ผู้ใช้สามารถโต้ตอบกับ Docker Daemon ผ่าน Client



โดยปกติแล้ว **Docker** ประกอบด้วย:

Docker Registry (Docker Hub): เปรียบเสมือน Store หรือคลังเก็บ Image ที่เราสามารถไปโหลดมาใช้ได้

× × × ×

x x x x

แล้ว

มันต่างกับ

ยังไฉละ???



Virtual Machine

The Heavyweight Approach:

- ต้องรัน Guest OS เต็มรูปแบบในทุกเครื่อง
- ใช้ทรัพยากร (RAM, CPU, Disk) สูง
- ใช้เวลานานในการ Boot เครื่อง (หลักนาที)
- มี Hypervisor เป็นตัวจัดการ Hardwares

x x x x



Docker C

The Lightweight Revolution:

- ใช้ OS Kernel ร่วมกับ Host (Shared Kernel)
- ไม่มี Guest OS ทำให้ขนาดเล็กและเบา
- Boot และพร้อมใช้งานทันที (หลักวินาที)
- ใช้ Docker Engine ในการจัดการรันไทม์

x x x x



สรุปข้อแตกต่าง VMs vs. Docker

× × × ×



หัวข้อเปรียบเทียบ

Virtual Machine (VM)

Docker Container

Operating System	มี Guest OS ในทุกๆ VM	ใช้ OS Kernel ร่วมกัน
Size	ใหญ่ (หลาย GB)	เล็ก (MB ถึงไม่กี่ร้อย MB)
Start Speed	ช้า (นาที)	เร็วมาก (วินาที)
Isolation	สูงมาก (Hardware Level)	สูง (Process Level)

Why

Using Docker?

x x x x



Standardization

สร้างมาตรฐานเดียวใน
การ Pack แอป ทำให้
ทุกคนทำงานบนพื้น
ฐานเดียวกัน



Portability

ย้ายแอปไปรันที่ไหนก็ได้ ไม่
ว่าจะเป็นเครื่องเพื่อน,
Cloud หรือ On-premise



Microservices

แยกแอปใหญ่ออกเป็น
ส่วนเล็กๆ รันแยกกัน
อิสระต่อกัน และแชร์
ทรัพยากรคุ้มค่า

- Get Docker
- What is Docker?
- Introduction
- Docker concepts
- Docker workshop
- Educational resources

Home / Get started

Get started

🗨 Ask Gordon 📄 Copy Markdown 📄 View Markdown

✎ Edit this page

🚩 Request changes

If you're new to Docker, this section guides you through the essential resources to get started.

Follow the guides to help you get started and learn how Docker can optimize your development workflows.

For more advanced concepts and scenarios in Docker, see [Guides](#).

Foundations of Docker

Install Docker and jump into discovering what Docker is.

📄 Get Docker

Choose the best installation path for your setup.

📄 What is Docker?

Learn about the Docker platform.

Learn the foundational concepts and workflows of Docker.

🚀 Introduction

Get started with the basics and the benefits of containerizing your applications.

🏠 Docker concepts

Gain a better understanding of foundational Docker concepts.

💻 Docker workshop

Get guided through a 45-minute workshop to learn about Docker.

Get Docker

What is Docker?

Introduction

Docker concepts

Docker workshop

Educational resources

Home / Get started / Get Docker

Get Docker

Ask Gordon

Copy Markdown

View Markdown

Edit this page

Request changes

Docker is an open platform for developing, shipping, and running applications.

Docker allows you to separate your applications from your infrastructure so you can deliver software quickly. With Docker, you can manage your infrastructure in the same ways you manage your applications.

By taking advantage of Docker's methodologies for shipping, testing, and deploying code quickly, you can significantly reduce the delay between writing code and running it in production.

You can download and install Docker on multiple platforms. Refer to the following section and choose the best installation path for you.

Docker Desktop terms

Commercial use of Docker Desktop in larger enterprises (more than 250 employees OR more than \$10 million USD in annual revenue) requires a [paid subscription](#).

Docker Desktop for Mac

A native application using the macOS sandbox security model that delivers all Docker tools to your Mac.

Docker Desktop for Windows

A native Windows application that delivers all Docker tools to your Windows computer.

Docker Desktop for Linux

A native Linux application that delivers all Docker tools to your Linux computer.

AI AND AGENTS

Overview

Docker Sandboxes

MCP Catalog and Toolkit Beta

Model Runner

Gordon

AI and Docker Compose

Docker Agent

APPLICATION DEVELOPMENT

Docker Desktop

Setup

Install

Mac

Mac permission requirements

Windows

Windows permission requirem...

Linux

VM or VDI environments

Sign in

Allowlist

Explore Docker Desktop

Features and capabilities

Settings and maintenance

Troubleshoot and support

Uninstall

Release notes

Docker Engine

Docker Offload

Docker Build

Docker Build Cloud

Home / Manuals / Docker Desktop / Setup / Install / Windows

Install Docker Desktop on Windows

 Ask Gordon

 Copy Markdown

 View Markdown

Docker Desktop terms

Commercial use of Docker Desktop in larger enterprises (more than 250 employees OR more than \$10 million USD in annual revenue) requires a [paid subscription](#).

This page provides download links, system requirements, and step-by-step installation instructions for Docker Desktop on Windows.

[Docker Desktop for Windows - x86_64](#)

[Docker Desktop for Windows - x86_64 on the Microsoft Store](#)

[Docker Desktop for Windows - Arm \(Early Access\)](#)

For checksums, see [Release notes](#)

Installation modes

Docker Desktop supports two installation modes. Per-user installation (Beta) is recommended for most users. It does not require administrator privileges to install or update, and the WSL 2 backend it uses covers the needs of the vast majority of Docker Desktop users.

	Per-user (recommended)	All users
Install location	%LOCALAPPDATA%\Programs\DockerDesktop	C:\Program Files\Docker\Docker
Registry keys	Current User (HKCU)	Local Machine (HKLM)

 Edit this page

 Request changes

Table of contents

Installation modes

System requirements

Install Docker Desktop on Windows

Install interactively

Install from the command line

Start Docker Desktop

Advanced system configuration and installation options

WSL: Verification and setup

Installer flags

Administrator privileges

Windows containers

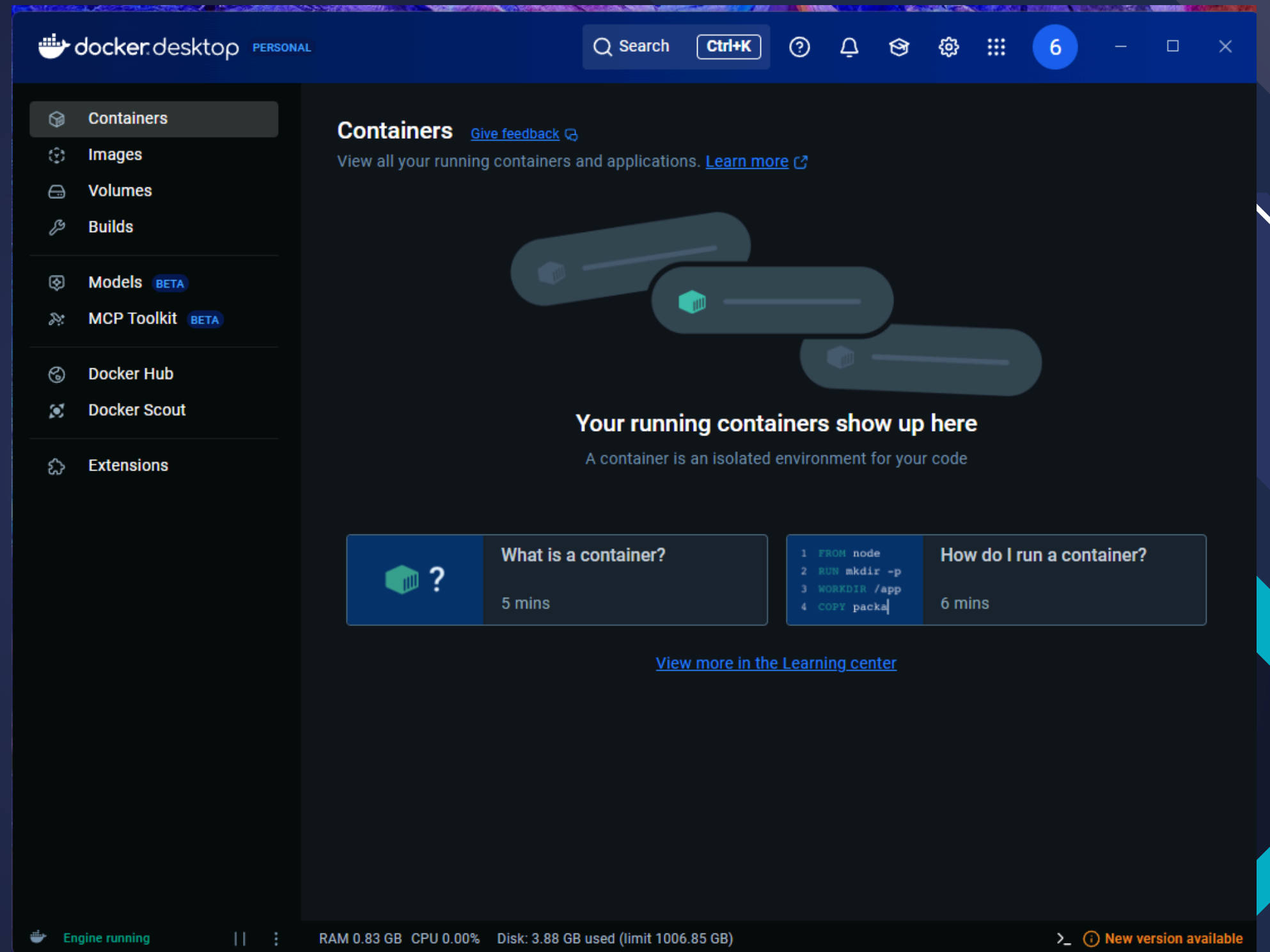
Where to go next



เปิดโปรแกรม Docker Desktop
ด้วย Gmail KMITL ให้เรียน

จุดเน้นย้ำ (Crucial Check _

- Windows: ต้องใช้ว่าติด (Windows Subsystem for Linux) 1
- Mac: แยกเวอร์ชันระหว่าง Intel และ Silicon (M1/M2/M3)



[Get Docker](#) | [Docker Docs](#)



คำสั่งเช็คความพร้อม: เมื่อที่
Terminal / PowerShell

docker

dot

เพื่อเช็คว่าบน OS ได้ติดตั้ง

```
PS C:\Users\จ๊อบ> docker -v  
Docker version 28.3.2, build 578ccf6
```

x x x x



มนตร์เสน่ห์ข้อแรก: "Hello World"

เรียนรู้คำสั่งแรก

docker run hello-

```
PS C:\Users\จ๊อบๆ > docker run hello-world
```

x x x x



- Docker จะหา Image ที่ชื่อ hello-world ในเครื่องก่อน (ซึ่งไม่มี)
- มันจะไปดาวน์โหลด (Pull) มาจาก Docker Hub อัตโนมัติ
- มันจะสร้าง Container แล้วพิมพ์ข้อความทักทายออกมาบนหน้าจอ

```
PS C:\Users\จู้บๆ > docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
Digest: sha256:96498ffd522e70807ab6384a5c0485a79b9c7c08ca79ba0862
Status: Downloaded newer image for hello-world:latest
```

```
Hello from Docker!
This message shows that your installation appears to be working c
```

x x x x



ลงมือทำของจริง: รัน Web Server (Nginx) ใน 1 นาที

คำสั่งรัน Web Server:

docker run -d -p :

อธิบายแะสูตรทีละตัวแป:

- d** (Detached mode): สั่งให้ทำงานเบื้องหลัง (Background) หน้าจอ Terminal จะได้ไม่ค้าง
- p 3000:80** (Port mapping): เชื่อมพอร์ต 3000 ของเครื่องเรา เข้ากับพอร์ต 80 ข้างใน Container
- name my-webserver**: ตั้งชื่อให้คอนเทนเนอร์นี้จำง่าย
- nginx**: ชื่อโปรแกรม/Image ที่เราจะเอามาใช้

ผลลัพธ์: ให้ผู้เรียนเปิดบราวเซอร์แล้วไปที่ _____ จะเจอหน้า Welcome ของ Nginx

x x x x



PS C:\Users\ ခုံပု > **docker** run -d -p 3000:80 --name my-app nginx

Unable to find image 'nginx:latest' locally

latest: Pulling from library/nginx

e95a6c7ea7d4: Downloading [=====>

] 6.225MB/29.79MB

df68ee7e7a00: Downloading [=====>

] 5.848MB/33.32MB

1b30016634d5: Download complete

acf093e7a04f: Download complete

cd9307c9ecd8: Download complete

fc6fd84b2a0: Download complete

1645c1e06f46: Download complete

x x x x



```
PS C:\Users\ຈຸ້ບຯ > docker run -d -p 3000:80 --name my-app nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
e95a6c7ea7d4: Pull complete
df68ee7e7a00: Pull complete
1b30016634d5: Pull complete
acf093e7a04f: Pull complete
cd9307c9ecd8: Pull complete
fcb6fd84b2a0: Pull complete
1645c1e06f46: Pull complete
Digest: sha256:ec4ed8b5299e5e90694af7750eb6dfffd2627317d30544d056b0371f8082f7bce
Status: Downloaded newer image for nginx:latest
c53e9486be30d0c28e757d8d290cb689c5beba639fbd8939d764e1edf4fea31b
```

x x x x

docker desktop

PERSONAL

Q Search

Ctrl+K

6

Containers

Images

Volumes

Builds

Models

MCP Toolkit

Docker Hub

Docker Scout

Extensions

Images

Give feedback

View and manage your local and Docker Hub images.

Learn more

LocalMy Hub

161.31 MB / 161.31 MB in use

2 images

Search

	Name	Tag	Image ID	Created	Size	Actions
☐	nginx	latest	9f33606b3685	14 days ago	161.3 MB	
☐	hello-world	latest	e2ac70e7319a	4 months ago	10.07 KB	

You're now signed in as

66070167@kmitl.ac.th

Engine running

RAM 0.00 GB CPU 0.00% Disk: -- GB used (limit -- GB)

> Terminal

New version available

Docker D

1.เข้าไปที่ Docker Desktop หน้าเมนู Image

2.ดูที่ Image ที่กำลังใช้งานอยู่

docker.desktop

PERSON

Containers

Images

Volumes

Builds

Models BETA

MCP Toolkit BETA

Docker Hub

Docker Scout

Extensions

Containers

[Give feedback](#)

[Learn more](#)

Container CPU usage

0.00% / 800% (8 CPUs available)

Container memory usage

7.7MB / 7.53GB

[Show charts](#)

Search

Only show running containers

	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	flamboyant_carson	25825631b2af	hello-world		0%	4 minutes ago	
☐	my-app	c53e9486be30	nginx	3000:80	0%	53 seconds ago	

Docke

1.เข้าไปที่ Docker Desktop หน้าเมนู Containers

2.ดูที่ Containers ที่กำลังทำงานงานอยู่

การติดตั้งและคำสั่ง

การติดตั้ง: Docker Desktop (Windows/Mac) หรือ Docker Engine (Linux)

คำสั่งที่ต้องรู้:

docker pull: โหลด Image จาก Registry

docker run: สร้างและรัน Container

docker ps: ดู Container ที่กำลังทำงานอยู่

docker stop / docker start: หยุดหรือเริ่ม Container

docker rm / docker rmi: ลบ Container หรือ Image

docker exec: การเข้าไปพิมพ์คำสั่งข้างใน Container



LAP101

Docker Basic

x x x x



ให้ผู้เรียนลองสร้าง
แสดงข้อมูลตัวเอง และเอาขึ้น
แล้วทำการ:

ง่ายๆ ด้วย

1. เขียน **Dockerfile**
2. **Build** เป็น **Image**
3. **Run** เป็น **Container** แล้วเปิดผ่าน **Browser**
4. ลองแก้ไขโค้ดแล้ว Build ใหม่



สร้างโปรเจกต์

ให้สร้าง Webserver ด้วย Node.js ด้วยคำสั่ง

npm init -y

และสร้างไฟล์ **index.js**
กับ **dockerfile** (ไม่มีนาทสกุล)

```
✓ WEEK_02
  > node_modules
  🐳 dockerfile
  JS index.js
  {} package-lock.json
  {} package.json
```

x x x x

JS index.js > ...

```
1  const express = require('express');
2  const app = express();
3  const PORT = 3000;
```

```
6  app.get('/', (req, res) => {
7    res.send(`
8      <h1 style="color: #333;">สวัสดีครับ/ค่ะ จากภายใน Docker Container</h1>
9      <p><b>ชื่อ-นามสกุล:</b> นายสมชาย สายโค้ด</p>
10     <p><b>รหัสนักศึกษา:</b> 67xxxxxx</p>
11    `);
12  });
```

```
14 app.listen(PORT, () => {
15   console.log(`Server is running on http://localhost:${PORT}`);
16
17 });
18
```

ภายในไฟล์ **index**
ให้เขียน code นี้ลงไป

← เปลี่ยนเป็นชื่อและรหัสของตัวเอง



สร้างไฟล์พิมพ์

ให้ผู้เรียนสร้างไฟล์ชื่อ `Dockerfile` ไว้ในโฟลเดอร์เดียวกับ
เพื่อเป็นคำสั่งในการประกอบ

..... ไว้ในโฟลเดอร์เดียวกัน

```
dockerfile > ...
1  # 1. ใช้ Base Image เป็น Node.js เวอร์ชัน 20 (Alpines จะมีขนาดเล็กและเบา)
2  FROM node:20-alpine
3
4  # 2. กำหนดโฟลเดอร์ทำงานหลักข้างใน Container
5  WORKDIR /app
6
7  # 3. ก๊อปปี้ไฟล์ package.json เพื่อเข้าไปติดตั้ง dependencies
8  COPY package.json ./
9
10 # 4. รันคำสั่งติดตั้ง Express
11 RUN npm install
12
13 # 5. ก๊อปปี้โค้ดทั้งหมดที่เหลือในเครื่องเรา เข้าไปใน Container
14 COPY . .
15
16 # 6. บอกว่าแอปนี้เปิดใช้งานพอร์ต 3000
17 EXPOSE 3000
18
19 # 7. คำสั่งที่ใช้เปิดรันตัวเว็บตอนจังหวะเปิด Container
20 CMD ["node", "index.js"]
21
```

การสร้าง Docker Image Dockerfile

x x x x



Dockerfile คืออะไร?: ไฟล์ที่เขียนคำสั่งเพื่อสร้าง Image

คำสั่งพื้นฐานใน D

- **FROM:** เลือก OS หรือ Runtime พื้นฐาน (เช่น node, python, alpine)
- **WORKDIR:** ตั้งค่าโฟลเดอร์ทำงาน
- **COPY / ADD:** คัดลอกไฟล์จากเครื่องเราเข้าไปใน Image
- **RUN:** สั่งรันคำสั่งตอน Build (เช่น npm install)
- **CMD:** คำสั่งที่จะให้ทำงานเมื่อ Container เริ่มต้น

การ Build: dock _ _ _ _

สั่ง และรันเช็คความถูกต้อง (Local Test)
ให้ผู้เรียนเปิด **Terminal/PowerShell** ในโฟลเดอร์นั้น แล้วใช้คำสั่งต่อไปนี้:

× × × ×

```
$ docker build -t student-web .  
[+] Building 1.1s (10/10) FINISHED  
=> [internal] load build definition from dockerfile 0.0s  
=> => transferring dockerfile: 983B 0.0s  
=> [internal] load metadata for docker.io/library/node:20-alpine 0.9s  
=> [internal] load .dockerignore 0.0s  
=> => transferring context: 2B 0.0s  
=> [1/5] FROM docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d50c 0.0s  
=> [internal] load build context 0.0s  
=> => transferring context: 40.35kB 0.0s  
=> CACHED [2/5] WORKDIR /app 0.0s  
=> CACHED [3/5] COPY package.json ./ 0.0s  
=> CACHED [4/5] RUN npm install 0.0s  
=> CACHED [5/5] COPY . . 0.0s  
=> exporting to image 0.0s  
=> => exporting layers 0.0s  
=> => writing image sha256:7a6c2a2774d92fab263959ec979af650544a69759ea399387d90168022dacac 0.0s  
=> => naming to docker.io/library/student-web 0.0s
```

(อย่าลืมนจุด . ด้านหลังสุด หมายถึงให้สร้างจากโฟลเดอร์ปัจจุบัน)

docker:desktop-linux

สั่งประกอบร่าง (Build Image):

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/l1r9tt3nfjx94ogq2xy9s5cxy

สั่ง run ขึ้นมาเพื่อทดสอบ: ด้วยคำสั่งต่อไปนี้

x x x x

docker run -

จ๊อบๆ

MINGW64 /w/IT/Server-side/TA/Week_02

```
$ docker run -d -p 3000:3000 --name my-app student-web  
745a0885e8ec3e08a3f6693d69d6faafd6e73b12e9938156e7dd5892ff19654d
```

จ๊อบๆ

MINGW64 /w/IT/Server-side/TA/Week_02

```
$ docker stop my-app && docker rm my-app  
my-app
```

ให้ผู้เรียนเปิดเว็บไปที่ <http://localhost:3000> ถ้าขึ้นชื่อและรหัสนักศึกษาแปลว่าสำเร็จ!

x x x x

เมื่อทดสอบเสร็จแล้ว ให้สั่งหยุดและลบตู้คอนเทนเนอร์เดิมออกก่อนเพื่อไม่ให้ซ้ำซ้อน:

dc

จากนั้นนำ image ขึ้น docker Registry (Docker Hub)
Loginเข้าสู่ Docker Hub ผ่าน Terminal ด้วยคำสั่ง:

(ให้ผู้เรียนกรอก Username และ Password ของ Docker Hub ที่สมัครไว้)

x x x x

ติดป้ายชื่อใหม่ (Tag) ให้ตรงกับชื่อบัญชี Docker Hub:
โครงสร้างคือ **docker tag [ชื่อเดิม] [username_ของคุณ]/[ชื่อ_image]**

ตัวอย่าง (เปลี่ยน somchai_hub เป็น username จริงของผู้เรียน)

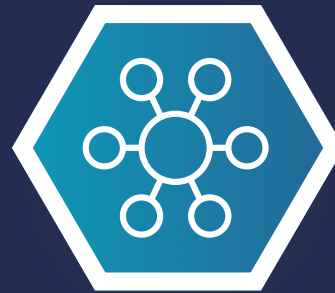
docker tag

ดันขึ้นคลาวด์ (Push):

dock

× × × ×

การจัดการ (Network)



Port Mapping

การเชื่อมต่อพอร์ตจาก
เครื่องเราเข้ากับ
Container



Environment Variables

การส่งค่าตัวแปร
เข้าไปในโปรแกรม



Docker Volumes

การทำ Data
Persistence เพื่อ
ให้ข้อมูลไม่หายเมื่อ
Container ถูกลบ



การจัดการ Container Docker

ปัญหา: ถ้าแอปเรามีทั้ง App, Database, และ Redis จะรันทีละตัวคงเหนื่อย

- **Docker Compos** การเขียนไฟล์ `docker-compose.yml` เพื่อสั่งรันทุกอย่างพร้อมกันด้วยคำสั่งเดียว
- **คำสั่งสำคัญ:** `docker-compose up -d` และ `docker-compose down`

x x x x



รายวิชา 06016418

×

×

×

×

×

×

×

×

การพัฒนาเว็บไซต์ฝั่งเซิร์ฟเวอร์